

Ollama 小白指南

把大模型装进自己电脑的"一键电饭煲"

疑问即目录 · 14 问图文解答 + 7 个动手实验 (全部实测通过)

学习日期 2026-10-01 · 环境: Windows 11 + RTX 4070 (12GB) + Ollama v0.35.0

开始之前: 3 分钟认识主角

Ollama 是一个让你在自己电脑上运行大语言模型的开源工具——一条命令下载模型，一条命令开始对话，还自带一个本地 API 服务器让别人写的 AI 应用连接它。名字来自 "llama" (羊驼，Meta 大模型 Llama 的吉祥物)，项目用 Go 语言写成，GitHub 星标十几万，是本地大模型领域事实上的入门标准。

云上大模型

按量付费 · 数据出户

VS

本机 Ollama

免费 · 断网可用 · 隐私

VS

裸用 llama.cpp

强大但全是命令行细节

图 0: Ollama 在三个世界中的位置——云服务的免费替代，llama.cpp 的易用外壳

? 疑问篇: 14 个问题逐个击破

1 Ollama 到底是什么? 一句话说清

Ollama = 模型下载器 + 推理引擎包装 + 本地 API 服务器, 三合一。你把它想象成大模型世界的"电饭煲": 米 (模型权重)、水 (你的问题)、电 (GPU/CPU) 放进去, 按一个键 (`ollama run`) 就能出饭 (回答)。至于电饭煲内部的加热曲线 (量化、显存调度、CUDA 调用), 它全部替你搞定。

严格地说: 它把开源推理内核 **llama.cpp** (以及新的自有引擎) 打包成单一服务进程, 配上模型仓库 (类 Docker 的 manifest/blob 结构)、命令行和 REST API。

2 它出现之前，本地跑大模型有多麻烦？

2023 年想在自己电脑上跑个 Llama，你需要：装 Python 环境 → 下载几十 GB 的原始权重 → 自己写脚本做格式转换 → 查文档选量化方案 → 手动编译 llama.cpp → 祈祷 CUDA 版本对得上。每一步都有坑，**整个流程像"自己种麦子磨面粉再蒸馒头"**。

Ollama 把这套流程压缩成两行命令：

```
ollama pull qwen3:0.6b    # 下载模型（自动选好量化格式）
ollama run qwen3:0.6b    # 直接开聊（自动配好显存）
```

它解决的问题不是"能不能跑"，而是**把门槛从"会编译 C++ 的工程师"降到"会开终端的任何人"**。

3 它是怎么做到的？整体架构长什么样

Ollama 在你电脑上其实是一个**常驻的服务进程 (server)** + 一个**客户端命令 (CLI)**。CLI 只是个"遥控器"，真正干活的是后台服务：

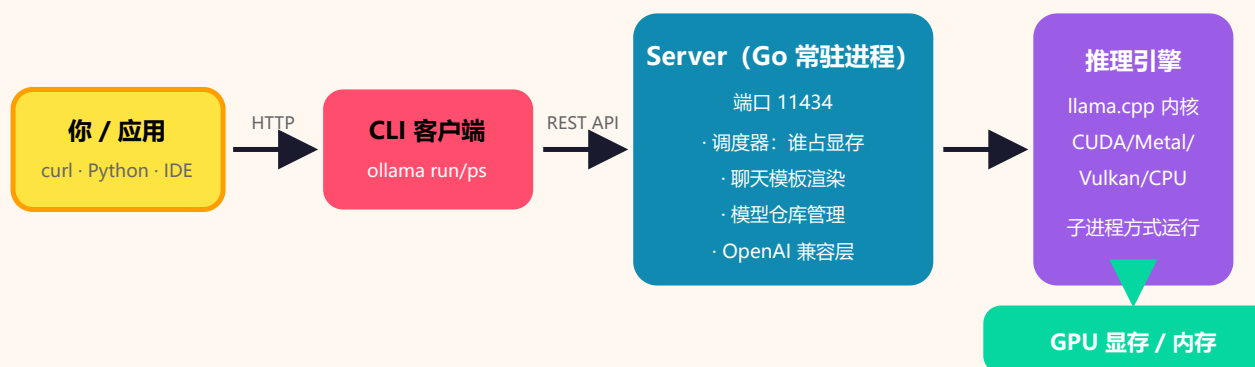


图 1：请求的旅行路线——CLI 和外部应用都只是发 HTTP 请求，server 调度推理引擎子进程干活

这个设计和 Docker 极像：**server** 相当于 Docker daemon，**CLI** 相当于 docker 命令，模型仓库结构也照搬了 Docker 的 manifest/blob（见疑问 5）。

💡 **知识库关联**：server 里那个“决定哪个模型常驻显存、哪个被换出”的**调度器**（server/sched.go），本质上就是[[构建流水线与Pass]]思想的运行时版本——把“加载→推理→卸载”拆成阶段管理。

4 模型文件到底是什么？为什么 8B 模型只有 5GB？

大模型的"知识"就是**几百上千个巨大的矩阵（一堆浮点数）**。模型文件就是把它们打包存储。**GGUF** 是 Ollama 生态的模型文件格式（llama.cpp 社区标准），单文件包含：权重 + 分词器 + 元数据（层数、上下文长度等）。

而文件能这么小的秘诀是**量化（quantization）**：训练时权重是 16 位浮点数（每个 2 字节），推理时可以压到 4 位整数（每个 0.5 字节）——就像把高清照片转成 JPEG，体积降 75%，画质轻微受损但完全可看：

8B 模型不同量化格式的体积（约）

FP16 原始精度 • 16GB • 16 位浮点（每个权重 2 字节）

Q8 • 8.5GB • 8 位整数

Q4_K_M • 4.9GB

← Ollama 默认：4 位整数，体积 1/3，质量损失很小

图 2：量化就是"有损压缩"——名字里的 Q4/Q8 就是每个权重占多少位

我们实测的 **qwen3:0.6b**（0.6B 参数、Q4 量化）只有 **522MB**——所以"本地跑大模型"的第一件事永远是：按显存选模型大小，而不是贪大。

5 models 目录里那一堆 sha256 文件是什么？（manifest + blobs）

Ollama 的模型存储完全**照搬 Docker 镜像的设计**：真正的数据叫 **blob**（二进制大对象），文件名就是它的 sha256 哈希值；一个 **manifest（清单）** 小 JSON 记录"这个模型由哪几个 blob 组成"。好处：两个模型共享同一层文件时**不用重复下载，还能顺带校验完整性**。

```
F:\ollama_models\
├── manifests\registry.ollama.ai\library\qwen3\0.6b  # 清单（几百字节）
└── blobs\
    ├── sha256-7f403014...  # 权重主体 522MB
    ├── sha256-ae370d88...  # 聊天模板 1.7KB
    ├── sha256-d18a5cc7...  # 许可证 11KB
    ├── sha256-cff3f395...  # 默认参数 120B
    └── sha256-b0830f4f...  # 元数据 490B
```

我们这次学习就**手动扮演了一回 ollama pull**：用 curl 按 registry 协议取清单 → 逐个下载 blob → 自己算 sha256 校验 → 放到对应路径。**ollama list** 立刻就认出了这个模型——"模型 = 清单 + 一堆哈希命名的文件"，没有半点魔法。

💡 **知识库关联**：用 sha256 校验每个文件再落盘，正是[[质检Gate与自我纠错循环]]里的客观 Gate——校验不过就整块作废，绝不让坏数据混进模型库。清单本身的内容寻址设计也体现了[[确定性脚本]]的精神：同样的哈希永远指向同样的内容。

6 ollama run 背后发生了什么？模型一直占着显存吗？

敲下 `ollama run qwen3:0.6b "你好"` 后：CLI 把请求发给 server → 调度器发现模型没加载 → **启动推理引擎子进程，把权重从磁盘读进显存** → 逐 token 生成回答 → 模型 **不立刻卸载**，默认在显存里多留 5 分钟（`OLLAMA_KEEP_ALIVE` 可改），等人再来问——下次提问就不用重新加载了。

实测对比（RTX 4070）：**首次提问 41 秒（含加载） vs 模型常驻后 1 秒**。用 `ollama ps` 能看到常驻状态：

NAME	ID	SIZE	PROCESSOR	CONTEXT	UNTIL
qwen3:0.6b	7df6b6e09427	930 MB	100% GPU	4096	4 minutes from now

调度器还负责多模型排队：显存够就并存，不够就把最久没用的换出。这就是为什么 Ollama 敢号称“并行服务多个请求”。

7 显存占用 930MB，模型文件才 522MB，多出来的 400MB 是什么？

多出来的主要是 **KV 缓存 (KV cache)**。模型生成每个字时，都要“回看”之前所有字——为了避免每次从头重算，中间计算结果（每个字的 Key/Value 矩阵）被缓存起来。**上下文越长，KV 缓存越大**，这是显存占用的第二大项（模型权重永远第一大）：



图 3: ``ollama ps`` 显示的 930MB = 权重 + KV 缓存 + 缓冲。把上下文从 4K 调到 32K，增长的主要是蓝色部分

推论：**显存不够时先砍上下文长度 (num_ctx)，比换小模型更划算。**

8 "Thinking..." 是什么？思考模式为什么值得开关

Qwen3 这类新一代模型是**混合推理 (hybrid thinking)** 模型：回答前可以先在一段特殊标记里"打草稿"——列思路、验算、自我纠正，然后再给出正式回答。Ollama 会把这段草稿识别出来单独显示（API 里放在 `thinking` 字段），并且**不计入正式回答**。

💡 思考段 (thinking tokens) : "用户问的是...我应该...验算一下 $17 \times 23 = 391$ ✓" — 慢但提升质量

正式回答: "391" — 干净利落

图 4：同一次生成里的两个阶段。API 的 `think` 参数可以关闭思考段换取速度

实测（同一台机器， 17×23 数学题）：开思考 2.9 秒（先打草稿）；关思考更快但 0.6B 小模型会开始胡言乱语。**结论：小模型更要开思考——容量不够，草稿来凑**；大模型简单任务可以关思考省时间。

9 temperature 和 seed 是什么？为什么温度 0 也偶尔不重复？

模型每一步输出的是"下一个字的概率表"（比如"奶"30%、"茶"25%...）。**temperature (温度)** 决定怎么从这张表里挑字：

温度 0：永远挑第一名（贪心）

奶 30%

茶 25%

咖 18%

→ 稳定、可复现，适合写代码/抽信息

温度 1.5：差距拉平，冷门词也有机会

奶 21%

茶 20%

咖 19%

→ 有创意、会跑偏，适合写文案/头脑风暴

图 5：同一个概率表，温度低=保守复读机，温度高=疯狂即兴演员

seed (随机种子) 固定了掷骰子的序列。实测：温度 0 + 固定种子写广告词，**连跑 3 次输出完全一致**（"一杯奶茶，温暖你的每一刻。"）；温度 1.5 同种子则给出完全不同的句子。但注意：温度 0 在 GPU 上**不是数学意义上的绝对确定**——并行浮点求和的顺序误差偶尔会让两个候选打平、发生分岔。要稳定结果，就 `temperature: 0 + seed 固定 + 保证硬件一致`。

💡 **知识库关联**：这正是[[确定性脚本]]在 AI 时代的延伸——LLM 天生随机，但"低温 + 固定种子"是给随机性上锁的标准手法，是构建可复现 AI 流水线的前提。

10 怎么用程序调用它？为什么有两个长得不一样的 API？

Ollama 同时提供两套 HTTP API（都在 `http://localhost:11434`）：

API	路径	谁在用
原生 API	<code>/api/chat</code> 、 <code>/api/generate</code> 、 <code>/api/embed</code>	Ollama 自家 CLI、需要 thinking 等独家功能时
OpenAI 兼容 API	<code>/v1/chat/completions</code> 等	海量现成工具（只需把 <code>base_url</code> 指到本机）

OpenAI 兼容层是 Ollama 普及的胜负手：全世界 AI 工具都按 OpenAI 的请求格式写，只要 Ollama “假装”自己是 OpenAI 服务器，这些工具就能零改造接入本地模型。实测把 `base_url` 指到 `localhost:11434/v1`，标准 OpenAI SDK 直接能用。

```
curl http://localhost:11434/v1/chat/completions -d '{
  "model": "qwen3:0.6b",
  "messages": [{"role": "user", "content": "say hi"}]
}' # 返回 OpenAI 格式: choices[0].message.content
```

11 Modelfile 是什么？怎么定制一个“专属人格”模型？

Modelfile 是模型的“配方文件”——**类比 Dockerfile 之于 Docker 镜像**。基于现成模型，叠加你自己的系统提示词、采样参数，`ollama create` 一下就得到一个新“模型”（其实只是新的 manifest 指向原有 blob，秒建不占空间）：

```
FROM qwen3:0.6b           # 基础模型
PARAMETER temperature 0.8  # 采样参数
SYSTEM 你是一位说话永远押韵的诗人助手。 # 系统提示词
```

实测 `ollama create poet -f Modelfile` 后问它“介绍一下春天”，它真的用诗体回答（“春天啊，万物复苏，/ 花儿在枝头绽放...”）。这个机制让“角色扮演/领域助手”可以像配置文件一样**进 git、可版本化、可分享**。

知识库关联：Modelfile 由 manifest+blobs 组合而成，与 Docker 镜像分层共享的思路同源——和[[构建流水线与Pass]]中“每阶段产物可复用”是同一个工程智慧。

12 Ollama、llama.cpp、LM Studio、vLLM 是什么关系？

名字	定位	一句话
llama.cpp	推理引擎（C/C++ 内核）	汽车的发动机，Ollama 的底层就是它
Ollama	易用外壳 + 模型仓库 + API	整车——发动机之上的方向盘和仪表盘
LM Studio	同类易用工具（闭源，带 GUI）	另一个牌子的整车，点点鼠标不用终端
vLLM	高吞吐服务框架	卡车车队——面向服务器批量并发，不吃个人 PC 生态

新版本 Ollama 也在自研引擎（源码 `ml/` 目录），并逐步把更多模型迁过去。**记住一条：跑起来选 Ollama，抠极致性能研究 llama.cpp，服务器规模化上 vLLM。**

13 我的电脑能跑多大的模型？（12GB 显存速查）

经验公式：**Q4 量化下，模型文件每 1B 参数约 0.6GB；显存还要额外装 KV 缓存（约几百 MB 到几 GB，看上下文长度）**。留出系统余量后（12GB 显存）：7~8B 模型 Q4 舒适运行；14B 勉强（要压上下文）；32B 基本没戏（除非 CPU+GPU 混合、忍受慢速）。本机 RTX 4070 实测参考：

模型规模	Q4 体积	12GB 显存	速度体感
0.5~2B（qwen3:0.6b）	0.5~1.5GB	轻松，可常驻多个	飞快（每秒上百 token）
7~8B（llama3.1:8b）	~4.9GB	舒适，上下文可开大	流畅（每秒几十 token）
14B（qwen3:14b）	~9GB	紧凑，小上下文	可用
32B+	20GB+	放不下，部分层落到内存	明显变慢

另外 Ollama 新增了云模型（`ollama run gpt-oss:120b-cloud` 这类带 `-cloud` 后缀的），本地只做转发、推理在云端跑——算“本地优先、云端兜底”的混合模式。

14 学会了能干什么？——这个项目给我的能力清单

🔒 **隐私对话**：日志、代码、私人文档丢给它分析，数据不出本机——断网也能用。

💧 **零成本后端**：给 LangChain/自写脚本当免费 LLM 后端（OpenAI 兼容接口换个 base_url 即可），实验随便烧 token。

🤖 **定制助手**：用 Modelfile 造“诗人/翻译/代码评审”等角色，SYSTEM 提示词进 git 版本管理。

🧪 **模型试吃**：HuggingFace 新模型出了 GGUF 就能 import 试玩，不满意 `ollama rm` 即走。

🏠 **理解 AI 基础设施**：量化、KV 缓存、调度、registry 协议——这些概念直接迁移到 vLLM、云 GPU 租用等一切推理场景。

⚡ **给 Claude Code 当备胎**：本地小模型接进编辑器补全/简单重构场景，主模型断网时不慌。

🧪 实验篇：7 个动手实验（本机全部实测通过）

以下每条命令都在本机（Windows 11 + Git Bash + Ollama 0.35.0 + RTX 4070）原样执行并核对过输出。**环境前提**：Ollama 已安装并运行（安装后托盘图标出现即表示 server 已在 11434 端口监听）。

⚠️ 本机环境坑（踩过实录）

- ① 本机代理（7890 端口）会拦截 localhost 请求 → curl 本机 API 必须 `--no-proxy '*'`;
- ② 代理 TUN 模式给域名分配假 IP (198.18.x.x) → `ollama pull` 直接失败（防 SSRF 检查拒绝），见实验 1 的手动替代方案；
- ③ 模型默认装 C 盘，C 盘满会下载中断（curl exit 23）→ 已用 `OLLAMA_MODELS=F:\ollama_models` 迁移；
- ④ 中文 JSON 直接写在命令行里可能因 GBK 编码乱码 → 写成 UTF-8 文件再 `--data-binary @文件`。

实验 1 看穿“模型”的本质：手动下载一个模型（理解 manifest + blobs）

目的：证明“模型只是清单+哈希文件”，顺便绕开代理坑。**操作**：

```
curl -s -H "Accept: application/vnd.docker.distribution.manifest.v2+json" \
  https://registry.ollama.ai/v2/library/qwen3/manifests/0.6b -o manifest.json
# ① 拿到清单：里面列着 5 个 blob 的 sha256 和大小
curl -sL -o weight.tmp https://registry.ollama.ai/v2/library/qwen3/blobs/sha256:7f403014...
sha256sum weight.tmp          # ② 自己校验：算出的哈希必须和文件名一致
# ③ 改名为 blobs/sha256-7f403014... 放进模型目录，清单放到
# manifests/registry.ollama.ai/library/qwen3/0.6b （完整脚本见 exercise\manual_pull.sh）
```


✅ **预期结果：** `ollama list` 直接列出 `qwen3:0.6b 7df6b6e09427 522 MB`——Ollama 完全认可"手工货"，因为它只认路径和哈希，不关心文件怎么来的。

实验 2 第一次对话 + 体会"模型常驻"

```
ollama run qwen3:0.6b "用一句话解释什么是本地大模型"
```

目的：跑通最小闭环，观察加载开销。**实现：**首次运行会加载模型到显存再生成；紧接着再跑一次同样命令。

✅ **预期结果：**第一次约 **40 秒**（含"Thinking..."草稿 + 模型加载），第二次**骤降到 1~2 秒**——这就是调度器把模型留在显存里等你（默认 5 分钟）。对比是理解"常驻缓存"最直接的方式。

实验 3 看显存账单：ollama ps

```
ollama ps
```

✅ **预期结果：** `qwen3:0.6b ... 930 MB 100% GPU 4096 4 minutes from now`——930MB 比文件 522MB 多出来的就是 KV 缓存等运行时开销（对应疑问 7）；"100% GPU"表示模型整个住进了显卡；"minutes from now"是卸载倒计时。

实验 4 思考模式开关对比（API + think 参数）

操作：把请求体存成 UTF-8 文件（避免中文乱码），对同一道数学题分别开/关思考：

```
# req.json: {"model": "qwen3:0.6b", "messages": [{"role": "user", "content": "计算 17*23 等于多少？只给数字。"}], "think": true, "stream": false}
curl -s --no-proxy '*' http://localhost:11434/api/chat --data-binary @req.json
# 再把 "think": true 改成 false 跑一次，对比响应里的 thinking 字段与耗时
```

✅ **预期结果：**think=true 时响应带 `thinking` 字段（草稿内容），正式回答准确；think=false 时无草稿、更快，但 0.6B 小模型可能输出跑题甚至乱码——亲眼看到"小模型需要思考兜底"。

实验 5 温度与种子：给随机性上锁

```
# 请求体加 "options": {"temperature": 0, "seed": 42}, 问"给一家奶茶店写一句广告词"
# 连跑 3 次；再把 temperature 改成 1.5 跑一次
```

✅ **预期结果：**温度 0 三次输出一字不差（实测："一杯奶茶，温暖你的每一刻。"×3）；温度 1.5 换了一副面孔（实测："一杯鲜甜，尽享幸福"）——这就是采样温度对"下一个字概率表"的重塑。

实验 6 OpenAI 兼容接口：让现成工具接入本地模型

```
curl http://localhost:11434/v1/chat/completions -d '{"model":"qwen3:0.6b",
  "messages":[{"role":"user","content":"say hi in 3 words"}]}'
# Python: 把 openai 库的 base_url 改成 http://localhost:11434/v1, api_key 随便填
```

✅ **预期结果：**返回标准 OpenAI 格式 `{"choices":[{"message":{"content":...}}]}`——任何支持自定义 `base_url` 的 AI 工具从此都能白嫖本地算力。

实验 7 Modelfile 造一个“诗人模型”

```
# Modelfile 三行：FROM qwen3:0.6b / PARAMETER temperature 0.8 / SYSTEM 你是一位说话永远押韵的诗人助手
ollama create poet -f Modelfile
ollama list                # 秒建，0 秒等待，几乎不占额外磁盘
ollama run poet "介绍一下春天"
```

✅ **预期结果：**`ollama create` 输出 `success`；`list` 里出现 `poet:latest`；对话输出自动变成诗体分段（实测：“春天啊，万物复苏，/ 花儿在枝头绽放...”）——SYSTEM 人设生效。

速查卡

命令 / 配置	作用
<code>ollama pull / run / rm <模型></code>	下载 / 对话 / 删除模型
<code>ollama list / ps / stop <模型></code>	已装模型 / 显存常驻状态 / 立即卸载
<code>ollama create <名> -f Modelfile</code>	用配方造自定义模型
<code>OLLAMA_MODELS=F:\ollama_models</code>	模型库存放位置（本机已设，模型在 F 盘）
<code>OLLAMA_KEEP_ALIVE=30m</code>	模型用完在显存里留守多久（默认 5m）
<code>OLLAMA_HOST</code> / 端口 11434	server 地址；API 文档见 <code>repo\docs/api.md</code>